



CodeSmile

Report di Modifica

RIFERIMENTO	ReportDiModifica
VERSIONE	1.0
DESTINATARIO	Prof. Andrea De Lucia Dott. Giammaria Giordano



Laurea Magistrale in Software Engineering & IT Management –Università di Salerno
Corso di Ingegneria del Software: tecniche avanzate
Prof. Andrea De Lucia, Dott. Giammaria Giordano

Gestione Gruppo

MEMBRO	EMAIL	MATRICOLA
MARTINA DE LUCIA	m.delucia18@studenti.unisa.it	NF22500053
ALESSANDRA RAIA	a.raia7@studenti.unisa.it	NF22500054
CARLA STEFANILE	c.stefanile1@studenti.unisa.it	NF22500097



Version

DATA	VERSIONE	DESCRIZIONE	AUTORI
22/01/2026	0.1	Prima Stesura	MDL, AR, CS
22/01/2026	0.2	Impact Analysis v.1	AR
22/01/2026	0.3	3.1 Dettagli dei fallimenti	CS
22/01/2026	0.4	Revisione e modifiche 3.1 Dettagli dei fallimenti	AR
23/01/2026	0.4	Processo di Manutenzione v.1	AR
30/01/2026	0.5	Capitolo Prompt Engineering	AR
13/02/2026	0.6	Impact Analysis v.2	MDL
13/02/2026	0.7	Code Smell Management	CS
15/02/2026	0.8	Code Smell Detection With LLM	MDL
15/02/2026	0.9	Stesura di Introduzione, 3, 3.1, 3.2, 3.3	MDL
15/02/2026	1.0	Seconda revisione	MDL



Sommario

Gestione Gruppo.....	2
Version	3
1 Introduzione.....	6
1.1 Riferimenti ad altri documenti	6
2 Impact Analysis	7
2.1 Obiettivi	7
2.2 Metodologia adottata.....	7
2.2.1 Metriche quantitative di impatto.....	8
2.3 Change Request CR00	8
2.3.1 SIS – CIS	8
2.3.2 AIS – DIS – FPS.....	9
2.3.3 Valutazione dell’impatto.....	10
2.4 Change Request CR01	10
2.4.1 SIS – CIS	10
2.4.2 AIS – DIS – FPS.....	11
2.4.3 Valutazione dell’impatto.....	12
2.5 Conclusioni	14
3 Processo di Manutenzione.....	15
3.1 Dettagli sul documento	15
3.2 CR00	15
3.2.1 Dettagli fallimenti e soluzioni adottate.....	16
3.3 CR01	31
3.4 Prompt Engineering.....	31
3.4.1 Architettura del sottosistema	32
3.4.2 Persistenza strutturale del catalogo LLM	32
3.4.3 Astrazione dei provider LLM	33
3.4.4 Orchestrazione della detection LLM	33
3.4.5 Normalizzazione e gestione dell’output LLM	33
3.4.6 Interfaccia grafica di Prompt Engineering.....	34
3.5 Code Smell Management.....	35
3.5.1 UC – Add Code Smell (Definizione e Validazione)	35
3.5.2 UC – Modify Code Smell (Evoluzione della Definizione)	35
3.5.3 UC – Remove Code Smell (Manutenzione del Catalogo).....	36



3.5.4 Interfaccia Grafica e State Management	36
3.6 Code Smell Detection With LLM.....	36
3.6.1 Principi di implementazione e retrocompatibilità	36
3.6.2 Modifiche introdotte nella GUI “Run”	37
3.6.3 Validazioni e mapping con UC01	37
3.6.4 Esecuzione della detection LLM e produzione output	38



1 Introduzione

Il presente documento descrive le attività svolte per l'evoluzione del progetto **CodeSmile** attraverso due Change Request: **CR00** e **CR01**. Il report raccoglie sia l'analisi dell'impatto delle modifiche sul sistema, sia la descrizione del processo di manutenzione adottato e delle soluzioni implementate, con l'obiettivo di fornire una visione tracciabile e verificabile dell'intervento effettuato.

Il documento è organizzato in due macro-sezioni:

- **Impact Analysis** (Capitolo 2): presenta obiettivi, metodologia e metriche adottate, e per ciascuna Change Request definisce SIS/CIS e la valutazione ex-post tramite AIS/DIS/FPIS, al fine di stimare l'impatto e il rischio di regressione.
- **Processo di Manutenzione** (Capitolo 3): descrive nel dettaglio l'origine delle modifiche, i fallimenti rilevati (per CR00) e le soluzioni implementate, oltre ai sottosistemi introdotti/estesi per CR01 (Prompt Engineering, Code Smell Management e integrazione della detection LLM nella GUI Run).

1.1 Riferimenti ad altri documenti

- **Change Request [ChangeRequest]**
Documento di riferimento che descrive le funzionalità additive proposte, le componenti coinvolte e i requisiti funzionali e non funzionali associati.
- **Test Case Specification Pre Modification [TestCaseSpecification-PreModification]**
Documento che esplicita il Category Partition sul sistema prima di effettuare le modifiche.
- **Test Incident Report Pre Modification [TestIncidentReport-PreModification]**
Documento che colleziona i risultati dei test effettuati.
- **Test Case Specification Post Modification [TestCaseSpecification-PostModification]**
Documento che esplicita il Category Partition sul sistema dopo aver effettuato le modifiche relative alla CR01.
- **Test Incident Report Post Modification [TestIncidentReport-PostModification]**
Documento che colleziona i risultati dei test effettuati.



2 Impact Analysis

Il presente documento di **Impact Analysis** analizza l'impatto architetturale e funzionale delle Change Request **CR00** e **CR01** previste per il progetto *CodeSmile*.

L'Impact Analysis rappresenta una fase fondamentale del processo di evoluzione del software e viene svolta con l'obiettivo di individuare i componenti potenzialmente coinvolti da una modifica e di valutare la propagazione dell'impatto all'interno del sistema.

In particolare, l'Impact Analysis ha lo scopo di:

- identificare le componenti del sistema direttamente coinvolte dalle Change Request;
- valutare la possibile propagazione dell'impatto verso altri moduli;
- stimare il rischio di regressioni e l'effetto sull'architettura esistente;
- supportare le decisioni progettuali e la pianificazione delle attività di implementazione e testing.

2.1 Obiettivi

Gli obiettivi del presente documento sono:

1. Identificare i componenti **direttamente coinvolti** da ciascuna Change Request (**SIS – Start Impact Set**).
2. Valutare i componenti **potenzialmente coinvolti in modo indiretto** (**CIS – Candidate Impact Set**).
3. Esplicitare l'insieme di **componenti effettivamente modificati** (**AIS – Actual Impact Set**), i componenti impattati ma non anticipati nel **CIS** e i componenti previsti come impattati ma non effettivamente modificati **FPIS**.
4. Stimare il **rischio architetturale** associato alle modifiche previste ed evidenziare le **dipendenze tra moduli** e i confini dell'impatto.
5. Fornire una **valutazione complessiva dell'Impact Analysis** a supporto della successiva fase di implementazione.

2.2 Metodologia adottata

L'Impact Analysis è stata condotta mediante:

- analisi statica della codebase di CodeSmile;
- ispezione della struttura architetturale (GUI, pipeline di analisi, orchestrazione, persistenza);
- tracciamento delle dipendenze funzionali tra componenti.

Per ciascuna Change Request sono stati individuati:

- **SIS (Start Impact Set)**: componenti direttamente coinvolte;
- **CIS (Candidate Impact Set)**: componenti potenzialmente coinvolte per propagazione;



Gli insiemi **AIS**, **FPIS** e **DIS** verranno individuati e analizzati per valutare la qualità dell'Impact Analysis.

2.2.1 Metriche quantitative di impatto

Per supportare l'analisi qualitativa con evidenze oggettive, la qualità dell'Impact Analysis viene valutata confrontando l'insieme dei componenti previsti come potenzialmente impattati (**CIS**) con l'insieme dei componenti effettivamente modificati in fase di implementazione (**AIS**). In particolare, si considerano: **FPIS** (False Positive Impact Set), ossia componenti previsti ma non modificati, e **DIS** (Discovered Impact Set), ossia componenti non previsti ma emersi durante l'implementazione.

Le metriche adottate sono:

- **Precision** = $|\text{CIS} \cap \text{AIS}| / |\text{CIS}|$
Misura quanto l'analisi sia *precisa*, ovvero quante componenti previste nel CIS risultino effettivamente modificate.
- **Recall** = $|\text{CIS} \cap \text{AIS}| / |\text{AIS}|$
Misura quanto l'analisi sia *completa*, ovvero quante componenti effettivamente modificate siano state previste in fase di impact analysis.

Un valore alto di Precision riduce sprechi (componenti "candidate" che poi non servono), mentre un valore alto di Recall riduce il rischio di sottostimare l'impatto (componenti scoperte in corso d'opera).

2.3 Change Request CR00

La Change Request **CR00** è di tipo **correttivo** e nasce dall'individuazione di incoerenze comportamentali nella modalità GUI di CodeSmile emerse durante i test di sistema.

La CR mira a correggere la validazione degli input e la gestione delle opzioni operative, senza introdurre nuove funzionalità né modificare la pipeline di analisi esistente.

2.3.1 SIS – CIS

SIS (Start Impact Set) – CR00

La CR00 è correttiva e mira a ripristinare la conformità della GUI rispetto alle specifiche di sistema, intervenendo su: validazione dei parametri numerici (es. max_workers), gestione delle combinazioni Multiple/Parallel/Resume e messaggistica coerente ("ignored without Multiple", ecc.).

Alla luce di tali requisiti, il **SIS** per CR00 include il componente responsabile della validazione e della preparazione dei parametri prima dell'avvio dell'analisi:

- `gui/code_smell_detector_gui.py`
(logica di validazione e gestione opzioni operative nella modalità GUI).

SIS(CR00) = {gui/code_smell_detector_gui.py}

CIS (Candidate Impact Set) – CR00

Pur essendo localizzata nella GUI, la correzione CR00 può propagarsi indirettamente verso moduli che:



1. **ricevono i parametri validati**, e
2. **gestiscono aspetti “di contorno”** legati alle modalità (es. Resume) e alla gestione output.

In particolare, la CR00 richiede coerenza end-to-end con le specifiche di sistema (suite test/gui_system_spec) e quindi è necessario includere nel CIS anche i componenti che orchestrano l’analisi e la generazione/merge dei risultati quando la GUI invoca l’esecuzione.

Si identifica pertanto il seguente **CIS**:

- gui/code_smell_detector_gui.py (validazione + opzioni)
- components/project_analyzer.py (orchestrazione analisi, in particolare casi legati a Resume e gestione flussi)
- utils/file_utils.py (gestione directory/output e merge risultati)

CIS(CR00) = {gui/code_smell_detector_gui.py, components/project_analyzer.py, utils/file_utils.py}

2.3.2 AIS – DIS – FPS

La presente sezione riporta l’**Actual Impact Set (AIS)** relativo alla Change Request **CR00**, determinato **a valle dell’implementazione** delle correzioni e della riesecuzione dei test di sistema.

L’obiettivo dell’analisi ex-post è verificare:

- se l’impatto reale è coerente con quanto previsto dall’Impact Analysis ex-ante;
- se sono emersi componenti inattesi (DIS);
- se sono presenti falsi positivi nell’analisi iniziale (FPIS).

A seguito dell’implementazione, l’**Actual Impact Set** della CR00 risulta composto dai seguenti moduli:

- **gui/code_smell_detector_gui.py**
(validazione input, gestione opzioni operative, messaggi di warning)
- **components/project_analyzer.py**
(gestione corretta dell’output in modalità Resume)
- **utils/file_utils.py**
(merge dei risultati preservando l’overview esistente)

L’impatto reale rimane **confinato ai moduli previsti**, senza coinvolgere la pipeline di analisi, i servizi di detection o i componenti di orchestrazione di livello inferiore.

L’analisi ex-post evidenzia che:

- **$AIS \subseteq CIS \cup SIS$**
- non sono emersi componenti inattesi (**$DIS = \emptyset$**)
- non sono presenti falsi positivi significativi (**$FPIS = \emptyset$**)



Le estensioni dell'impatto rispetto al SIS iniziale sono motivate dal rafforzamento della gestione dell'output in modalità Resume e sono coerenti con le dipendenze architetturali del sistema.

2.3.3 Valutazione dell'impatto

Valutazione qualitativa

L'impatto della CR00 è **basso e localizzato**: la correzione riguarda la fase pre-pipeline (validazione/gestione opzioni) e piccoli adeguamenti di orchestrazione e gestione output, mantenendo invariati i servizi di detection e la pipeline principale di analisi. Questo è coerente con l'obiettivo della CR00 di riallineare la GUI alle specifiche senza introdurre nuove funzionalità e senza alterare il core.

Il principale rischio di regressione è concentrato su:

- **coerenza dei messaggi** attesi dai test di sistema,
- **casi limite** (es. modalità Multiple con un solo progetto, Resume ignorato in Parallel, ecc.).

Valutazione quantitativa (Precision/Recall)

Assumendo come CIS l'insieme definito nella sezione 2.3.1 e come AIS quello osservato in implementazione, per CR00 risulta che i componenti modificati ricadono nei componenti previsti, con assenza di componenti scoperte (DIS) e assenza di falsi positivi (FPIS). In questo scenario:

- **Precision = 1.00**
- **Recall = 1.00**

2.4 Change Request CR01

La Change Request CR01 è di tipo **additivo** e introduce un'estensione opzionale della modalità GUI di CodeSmile per abilitare una detection LLM "prompt-driven", senza modificare il comportamento della pipeline standard. L'utente può: abilitare/disabilitare LLM-mode, scegliere il provider (locale/API), selezionare smell per LLM, gestire prompt (draft/default) e gestire il catalogo smell tramite due nuove interfacce dedicate (Prompt Engineering e Gestione Code Smell), supportate da una persistenza strutturata (catalogo smell/prompt/provider).

I requisiti funzionali FR1–FR8 e non funzionali (testabilità, estendibilità, robustezza, usabilità, tracciabilità), definiti nel documento di Change Request, guidano l'identificazione dei componenti impattati e dei principali rischi architetturali.

2.4.1 SIS – CIS

SIS (Start Impact Set) – CR01

Sulla base degli obiettivi della CR01 (abilitazione LLM in Run GUI, Prompt Engineering, Gestione Code Smell, persistenza e disaccoppiamento via astrazione/Bridge), il **SIS** include:



- `gui/code_smell_detector_gui.py`
(estensione GUI Run: toggle LLM, selezione provider/smell, invocazione orchestrazione LLM e salvataggio risultati)

SIS(CR01) = {gui/code_smell_detector_gui.py}

CIS (Candidate Impact Set) – CR01

CR01 specifica che la modalità LLM deve essere **opzionale** e che la pipeline standard non deve subire modifiche comportamentali. Tuttavia, l'introduzione di un nuovo flusso in GUI e di una fase aggiuntiva di analisi post-pipeline richiede di considerare nel CIS anche i componenti della baseline che possono essere coinvolti indirettamente per **integrazione e regressione**, in quanto attraversati dal flusso di analisi standard prima dell'eventuale invocazione della modalità LLM (UC01).

In particolare, sono candidati all'impatto (in termini di testing e verifiche di non-regressione):

- **components/project_analyzer.py** (orchestrazione analisi standard, gestione output e flussi di esecuzione)
- **components/inspector.py** e **components/rule_checker.py** (core detection statica e regole)
- **moduli di output/reporting** e utility collegate (es. generazione/aggiornamento di `overview.csv`), per verificare che l'attivazione della modalità LLM non alteri i risultati statici o il formato di output.

Si definisce pertanto:

CIS(CR01) = {gui/code_smell_detector_gui.py, components/project_analyzer.py, components/inspector.py, components/rule_checker.py, moduli di output/reporting}

2.4.2 AIS – DIS – FPS

La presente sezione riporta l'Actual Impact Set (AIS) relativo alla Change Request CR01, determinato a valle dell'implementazione dell'estensione LLM e della verifica tramite suite di test.

L'obiettivo dell'analisi ex-post è verificare:

- se l'impatto reale è coerente con quanto previsto (SIS/CIS);
- se sono emersi componenti inattesi (**DIS**);
- se sono presenti falsi positivi nell'analisi iniziale (**FPIS**).

Definizione operativa adottata

In questa analisi, si considera "impattato" un artefatto (file/modulo) che sia stato **modificato (Modified)**, **aggiunto (Added)** o **rimosso (Deleted)**. Pertanto:

AIS = Modified $\cup$ Added $\cup$ Deleted

AIS(CR01)

A seguito dell'implementazione della CR01, l'Actual Impact Set risulta composto da:

Modified (baseline):



- `gui/code_smell_detector_gui.py`
(estensione controlli GUI Run: attivazione LLM-mode, selezione provider/smell, invocazione analisi LLM, integrazione con salvataggio risultati)

Added (nuove componenti introdotte dalla CR01):

- `package llm_detection/*`
(orchestrazione LLM, gestione provider locale/API, normalizzazione risultati, servizi di catalogo)
- `package prompt_engineering/*`
(interfaccia e logica per gestione prompt draft/default e test prompt su cartelle)
- `gui/manage_code_smells_gui.py`
(GUI per CRUD smell e gestione informazioni associate)
- `config/llm_catalog.json`
(persistenza del catalogo smell/prompt/provider)

Non sono state rilevate rimozioni di componenti (**Deleted** = $\emptyset$).

DIS(CR01) e FPIS(CR01)

Coerentemente con la metodologia adottata (CIS definito sulla baseline), i moduli **Added** introdotti dalla CR01 non erano presenti nella baseline e pertanto non potevano essere inclusi nel CIS come file esistenti. Ne consegue che:

- **DIS(CR01) = AIS(CR01) – CIS(CR01)**
→ include le nuove componenti introdotte dalla CR01 (sottosistemi `llm_detection`, `prompt_engineering`, GUI gestione smell e catalogo JSON).

D'altro canto, alcuni moduli baseline inclusi nel CIS per ragioni di regressione possono non essere stati modificati. Pertanto:

- **FPIS(CR01) = CIS(CR01) – AIS(CR01)**
→ comprende i componenti della pipeline standard e reporting inclusi nel CIS a scopo di verifica di non-regressione (`analyzer/inspector/rule_checker` e `output`), che non sono stati oggetto di modifica.

In sintesi, l'impatto reale della CR01 risulta **prevalentemente additivo** (introduzione di nuovi moduli), mentre l'impatto sul core della pipeline standard si manifesta principalmente come necessità di testing e verifica di integrazione.

2.4.3 Valutazione dell'impatto

Valutazione qualitativa

L'impatto della CR01 è **medio**: pur non introducendo modifiche alla pipeline standard, la CR aggiunge un sottosistema dedicato alla detection LLM e due nuove interfacce (Prompt Engineering e Gestione Code Smell), introducendo nuove responsabilità architetturali (persistenza catalogo, orchestrazione, integrazione provider e normalizzazione risultati).



L'impatto è tuttavia controllato poiché la nuova modalità è **opzionale** e si innesta come fase aggiuntiva rispetto alla detection statica, mantenendo separati i ruoli tra:

- GUI di esecuzione (configurazione e lancio),
- sottosistema LLM (orchestrazione e provider),
- persistenza (catalogo),
- GUI di gestione (prompt/smell).

I principali rischi di regressione si concentrano su:

- coerenza dell'output e compatibilità con la modalità standard (overview/report);
- consistenza del catalogo smell/prompt/provider (ID unici e gestione CRUD);
- robustezza del flusso LLM (latenza, fallimenti provider, validazioni input).

Valutazione quantitativa (Precision/Recall)

La valutazione quantitativa è influenzata dalla natura **additiva** della CR01 e dalla definizione del **CIS**, costruito sui componenti **baseline** potenzialmente soggetti a regressione. Poiché l'**AIS** include anche le componenti **Added**, l'applicazione diretta delle metriche sull'intero AIS produce valori penalizzati "per costruzione": i nuovi moduli Added ricadono inevitabilmente in **DIS = AIS – CIS**.

Per ottenere una lettura corretta della qualità dell'Impact Analysis, si distinguono due perimetri di valutazione.

1) Valutazione sui componenti baseline (accuratezza della previsione ex-ante sui file pre-esistenti)

Si considerano:

- **AIS_baseline** = **AIS** $\cap$ **Baseline** (solo componenti baseline effettivamente modificati o rimossi)
- **CIS_baseline** = **CIS** (componenti baseline candidati all'impatto per regressione/integrazione)

Nel caso della CR01:

- **AIS_baseline** coincide con **{gui/code_smell_detector_gui.py}**;
- **FPIS(CR01)** $\neq \emptyset$, poiché parte dei componenti baseline inclusi nel CIS a scopo cautelativo (es. analyzer/inspector/rule_checker e output/reporting) non risultano modificati.

Ne consegue che, rispetto alla baseline:

- il **Recall_baseline** risulta **pari a 1.00**, in quanto l'unico componente baseline realmente modificato era incluso nel CIS;
- la **Precision_baseline** risulta **< 1.00**, poiché esistono falsi positivi (FPIS non vuoto), dovuti alla scelta di includere nel CIS componenti baseline per coprire rischi di non-regressione che non si sono materializzati in modifiche.



2) Valutazione complessiva includendo i moduli Added

Estendendo la valutazione all'intero AIS (baseline + Added), si osserva che:

- il **Recall** risulta necessariamente < 1.00 , poiché DIS include i moduli introdotti dalla CR01 (Added), non presenti nella baseline e quindi non includibili nel CIS pre-implementazione;
- anche la **Precision** risulta < 1.00 perché il CIS contiene componenti baseline non modificati (FPIS).

In questa impostazione, FPIS e DIS non indicano una scarsa qualità dell'Impact Analysis, ma riflettono rispettivamente:

- la **conservatività** del CIS (copertura preventiva della non-regressione sulla pipeline standard),
- la **dimensione additiva** della CR01 (introduzione di nuove componenti).

2.5 Conclusioni

L'Impact Analysis condotta per CR00 e CR01 evidenzia due profili di impatto differenti:

- **CR00** presenta un impatto **basso e localizzato**, limitato alla GUI e a moduli di orchestrazione/output necessari a garantire coerenza con le specifiche e con i test di sistema. L'analisi ex-post mostra assenza di componenti inattesi ($DIS = \emptyset$) e assenza di falsi positivi significativi ($FPIS = \emptyset$), con metriche massimizzate (Precision e Recall pari a 1).
- **CR01** presenta un **impatto medio e prevalentemente additivo**: l'estensione LLM introduce nuove componenti (orchestrazione, provider, persistenza del catalogo, nuove GUI) mantenendo invariata la pipeline standard. In linea con la scelta metodologica adottata (CIS definito sulla baseline e AIS comprensivo anche di componenti Added), l'analisi ex-post evidenzia un **DIS** associato ai nuovi moduli introdotti. I componenti della pipeline standard inclusi nel CIS a scopo di non-regressione (**components/project_analyzer.py**, **components/inspector.py**, **components/rule_checker.py** e i **moduli di output/reporting**) **non risultano modificati** e pertanto rientrano nel **FPIS**, rappresentando falsi positivi attesi dovuti a un CIS definito in modo conservativo per garantire l'assenza di regressioni.

Nel complesso, l'analisi fornisce una delimitazione chiara dell'impatto e supporta la pianificazione delle attività di implementazione e testing, distinguendo correttamente tra correzioni localizzate (CR00) ed estensioni architetturali additive (CR01).



3 Processo di Manutenzione

Il presente capitolo descrive il **processo di manutenzione** adottato per l'implementazione delle Change Request **CR00** e **CR01**, con l'obiettivo di collegare in modo operativo i risultati dell'Impact Analysis ([Capitolo 2](#)) alle attività di correzione/estensione effettivamente svolte sulla codebase.

Nel complesso, il capitolo 3 fornisce una descrizione tracciabile delle modifiche: dalla motivazione (fallimenti o requisiti), alle decisioni implementative, fino agli effetti osservabili sul comportamento del sistema.

3.1 Dettagli sul documento

Il capitolo è organizzato come segue:

- **Sezione 3.2 – CR00:** descrive l'origine della CR correttiva a partire dai **fallimenti rilevati nei test di sistema**. Per ciascun test fallito vengono riportati: il nome del test, l'errore osservato, la causa (root cause funzionale e/o progettuale) e la soluzione implementata, evidenziando come la modifica ripristini la conformità alle specifiche senza introdurre nuove funzionalità.
- **Sezione 3.3 – CR01:** introduce la CR additiva e ne sintetizza lo scopo, ossia l'estensione opzionale della modalità GUI per supportare una detection **LLM prompt-driven** mantenendo invariata la pipeline standard. La sezione funge da inquadramento per le sottosezioni successive, che dettagliano i sottosistemi e le funzionalità introdotte.
- **Sezione 3.4 – Prompt Engineering (UC02):** illustra l'implementazione dell'ambiente di sperimentazione dei prompt smell-specific, comprensivo di gestione draft/default, test iterativo su progetti reali e salvataggio dell'output (incluso raw output), con attenzione alle scelte architetturali di isolamento e testabilità.
- **Sezione 3.5 – Code Smell Management:** descrive il modulo di gestione del catalogo degli smell (UC di aggiunta, modifica e rimozione), evidenziando i vincoli di consistenza (ID immutabili, validazioni) e il ruolo della persistenza centralizzata del catalogo come fonte di verità condivisa.
- **Sezione 3.6 – Code Smell Detection With LLM (UC01):** dettaglia l'integrazione della detection LLM nella GUI "Run", includendo i controlli introdotti, le validazioni previste dallo Use Case, l'orchestrazione tramite provider e l'esportazione dei risultati in un report dedicato, preservando la retrocompatibilità e la non-regressione del flusso standard.

3.2 CR00

La Change Request **CR00** è una richiesta di manutenzione **correttiva** emersa a seguito dell'esecuzione della suite di **test di sistema** sulla versione pre-modifica dell'applicazione. In particolare, alcuni test della specifica di sistema relativi alla modalità GUI "Run" hanno evidenziato comportamenti non conformi alle aspettative, principalmente riconducibili a: (i) **validazione non uniforme** di parametri numerici, e (ii) **gestione incompleta o incoerente** di combinazioni tra opzioni operative (es. Multiple/Parallel/Resume) e relativa messaggistica di warning/errore.



L'obiettivo della CR00 è quindi ripristinare la conformità della GUI alle specifiche di sistema, mantenendo invariata la pipeline di analisi esistente e senza introdurre nuove funzionalità. In questo senso, l'intervento si concentra sul livello di **controllo e orchestrazione** dell'esecuzione, assicurando che configurazioni non valide vengano intercettate prima dell'avvio dell'analisi e che l'utente riceva messaggi coerenti con la specifica.

3.2.1 Dettagli fallimenti e soluzioni adottate

Nome del test:	test_TC_1_5
Errore riportato:	Nessun messaggio di errore visualizzato; l'analisi prosegue ignorando il parametro "Number of Walkers".
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando analizza una cartella che contiene un file Python con più code smell con "Multiple", "Parallel" e "Resume" non attive e -1 nel campo "Number of Walkers". In particolare, il test verifica che: - il parametro "Number of Walkers" venga validato anche quando la modalità "Parallel" non è attiva; - un valore ≤ 0 venga considerato non valido; - venga prodotto un messaggio di errore e l'analisi non venga avviata in presenza di un parametro numerico non corretto. Il fallimento è dovuto a: Root cause funzionale: Il parametro NW1 (numero di workers ≤ 0) non viene validato perché la modalità parallel non è attiva. Di conseguenza, il valore di max_workers viene ignorato e non genera alcuna eccezione. Root cause di progettazione: Il controllo sui parametri dipendenti dalla modalità di esecuzione non è uniforme: max_workers viene validato solo se parallel è attivo. Conclusione: Il test fallisce perché il controllo di validità su max_workers è implicitamente legato alla modalità parallela, mentre il test assume una validazione indipendente dalla configurazione di esecuzione.	
Soluzione:	
Introdotta una validazione esplicita dei parametri numerici prima dell'esecuzione dell'analisi, viene visualizzato: "Error: max_workers must be greater than 0".	

Nome del test:	test_TC_1_8 / test_TC_1_18
Errore riportato:	Durante la seconda esecuzione in modalità "Multiple" con "Resume" attivo, vengono analizzati i progetti non ancora presenti nel file execution_log.txt, ma i risultati delle analisi precedenti risultano persi e non viene prodotto alcun merge degli output.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando analizza più progetti in modalità "Multiple" con "Resume" attivo, in presenza di un file execution_log.txt che registra parzialmente i progetti già analizzati. In particolare, il test verifica che:	



- i progetti già presenti nel file di log vengano correttamente esclusi dall'analisi;
- i risultati prodotti nelle esecuzioni precedenti vengano preservati e integrati con quelli della nuova analisi.

Il fallimento è dovuto a:

- **Root cause funzionale:**

Il sistema applica correttamente la logica di esclusione dei progetti già registrati nel file `execution_log.txt`, ma non mantiene i risultati precedentemente prodotti, causando la perdita degli output relativi ai progetti già analizzati.

- **Root cause di progettazione:**

Il componente `project_analyzer` esegue una pulizia della directory di output all'avvio dell'analisi, comportamento incompatibile con la semantica della modalità "Resume", che richiederebbe la persistenza e l'integrazione dei risultati esistenti.

Nel componente `utils/file_utils.py` Il metodo `merge_results()` raccoglieva solo i CSV dalla cartella `project_details/` (i nuovi progetti analizzati) e sovrascriveva completamente `overview.csv`, perdendo i risultati precedenti.

Conclusione:

Il test fallisce perché la modalità "Resume" è implementata solo parzialmente: il meccanismo di esclusione dei progetti funziona correttamente, mentre la gestione dello stato e dei risultati pregressi non è supportata.

Soluzione:

Modificato il comportamento di inizializzazione del componente `project_analyzer` evitando la pulizia della directory di output quando "Resume" è attivo; nel componente `file_utils.py` prima di fare il merge, il metodo controlla se esiste già un `overview.csv` e lo include nella concatenazione, preservando i risultati dei progetti precedentemente analizzati.

Nome del test:	test_TC_1_19
Errore riportato:	Premendo il pulsante di avvio dell'analisi non viene visualizzato alcun messaggio nell'interfaccia; nel terminale viene sollevata l'eccezione ValueError: invalid literal for int() with base 10: 'a'.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando viene inserito un valore non numerico nel campo "Number of Walkers". In particolare, il test verifica che: • l'input venga validato prima dell'avvio dell'analisi; • un valore non numerico produca un messaggio di errore visibile all'utente; • l'analisi non venga avviata in presenza di parametri non validi. Il fallimento è dovuto a: • Root cause funzionale: Il valore inserito nel campo "Number of Walkers" viene passato direttamente alla conversione numerica senza alcuna validazione preventiva. • Root cause di progettazione: L'eccezione sollevata durante la conversione non viene intercettata a livello applicativo e non viene propagata all'interfaccia grafica. Conclusione: Il test fallisce perché un input non valido provoca un'eccezione non gestita, con conseguente assenza di feedback per l'utente e mancata segnalazione dell'errore.	



Soluzione:

Introdotta una validazione esplicita sugli input numerici prima della conversione e gestire l'eccezione mostrando un messaggio di errore coerente nell'interfaccia: "Error: max_workers must be a number".

Nome del test:	test_TC_1_20
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità multipla parallela senza resume nonostante le modalità "Multiple", "Parallel" e "Resume" siano attive in presenza di più un progetto contenenti un solo file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando "Multiple", "Parallel" e "Resume" sono attive, in presenza di più progetti contenenti un solo file Python. In particolare, il test verifica che: • venga gestita l'incompatibilità tra modalità; • l'utente venga informato del comportamento effettivamente adottato dal sistema. Il fallimento è dovuto a: • Root cause funzionale: La modalità "Resume" viene ignorata durante l'esecuzione. • Root cause di progettazione: L'incompatibilità tra "Parallel" e "Resume" non viene comunicata all'utente tramite alcun messaggio. Conclusione: Il test fallisce perché il sistema degrada la configurazione in modo silenzioso, senza rendere esplicito il comportamento applicato.	
Soluzione:	
Implementato un controllo esplicito della combinazione tra "Parallel" e "Resume" che, in caso di attivazione congiunta, mantenga la modalità parallela, ignori "Resume" e notifichi all'utente il comportamento adottato: • quando Parallel=True e Resume=True insieme, stampa il warning "Warning: in parallel mode, resume mode is ignored"; • resume viene ignorato (analyze_projects_parallel non supporta resume); • l'utente è informato che il suo setting Resume non ha effetto.	

Nome del test:	test_TC_1_21
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità multipla parallela senza resume nonostante le modalità "Multiple", "Parallel" e "Resume" siano attive in presenza di più un progetto contenenti più file di cui almeno uno Python.
Causa del problema:	
Il test intende verificare il comportamento del sistema in modalità "Multiple" e "Parallel" con "Resume" attivo, in presenza di più progetti contenenti più file di cui almeno uno Python. Il test verifica che:	



- i progetti già analizzati vengano esclusi;
- l'utente venga informato dell'eventuale incompatibilità tra modalità.

Il fallimento è dovuto a:

- **Root cause funzionale:**

La modalità "Resume" non viene applicata.

- **Root cause di progettazione:**

L'assenza di messaggi informativi impedisce all'utente di comprendere il comportamento reale del sistema.

Conclusione:

Il test fallisce perché il sistema non rende trasparente la gestione delle modalità incompatibili.

Soluzione:

Implementato un controllo esplicito della combinazione tra "Parallel" e "Resume" che, in caso di attivazione congiunta, mantenga la modalità parallela, ignori "Resume" e notifichi all'utente il comportamento adottato:

- quando Parallel=True e Resume=True insieme, stampa il warning "Warning: in parallel mode, resume mode is ignored";
- resume viene ignorato (analyze_projects_parallel non supporta resume);
- l'utente è informato che il suo setting Resume non ha effetto.

Nome del test:	test_TC_1_22
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita con la modalità "Multiple" attiva in presenza di un progetto contenente un file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando viene attivata la modalità "Multiple" in presenza di un solo progetto contenente un file Python. In particolare, il test verifica che: • l'uso della modalità "Multiple" venga bloccato o segnalato come non valido; • l'analisi non venga avviata con una configurazione incoerente. Il fallimento è dovuto a: • Root cause funzionale: La modalità "Multiple" viene attivata anche con un solo progetto. • Root cause di progettazione: Non è presente alcun controllo sul numero minimo di progetti richiesto per abilitare la modalità "Multiple". Conclusione: Il test fallisce, non rilevando i code smell, perché una configurazione logicamente non valida non viene intercettata dal sistema.	
Soluzione:	
Introdotta una validazione preliminare che impedisca l'attivazione della modalità "Multiple" quando il numero di progetti selezionati è inferiore a due, bloccando l'avvio dell'analisi e visualizzando un messaggio esplicativo: • conta i progetti (directory) nella cartella di input; • esclude le directory speciali "output" e "execution_log.txt"; • se ci sono meno di 2 progetti, stampa l'errore "Multiple mode isn't available with only one project" e interrompe (return);	



- previene l'esecuzione di analisi inutili in Multiple mode.

Nome del test:	test_TC_1_23
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita nonostante le modalità "Multiple" e "Parallel" siano attive in presenza di un progetto contenente un file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando viene attivata la modalità "Multiple" e "Parallel" in presenza di un solo progetto contenente un file Python. In particolare, il test verifica che: • l'uso della modalità "Multiple" venga bloccato o segnalato come non valido; • la combinazione con "Parallel" non venga accettata con un solo progetto. Il fallimento è dovuto a: • Root cause funzionale: Il sistema consente l'esecuzione (errata) dell'analisi anche quando "Multiple" è attivo con un solo progetto. • Root cause di progettazione: Non è implementato alcun controllo di coerenza tra numero di progetti e modalità "Multiple" e "Parallel". Conclusione: Il test fallisce perché una configurazione non valida viene accettata senza segnalazioni e porta ad un'analisi fallimentare.	
Soluzione:	
Introdotta una validazione preliminare che impedisca l'attivazione della modalità "Multiple" quando il numero di progetti selezionati è inferiore a due, bloccando l'avvio dell'analisi e visualizzando un messaggio esplicativo: • conta i progetti (directory) nella cartella di input; • esclude le directory speciali "output" e "execution_log.txt"; • se ci sono meno di 2 progetti, stampa l'errore "Multiple mode isn't available with only one project" e interrompe (return); • previene l'esecuzione di analisi inutili in Multiple mode.	

Nome del test:	test_TC_1_24
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita nonostante le modalità "Multiple" e "Resume" siano attive in presenza di un progetto contenente un file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando "Multiple" e "Resume" sono attivi in presenza di un solo progetto contenente un file Python. In particolare, il test verifica che: • la configurazione venga segnalata come non valida; • l'analisi non venga avviata. Il fallimento è dovuto a:	



- **Root cause funzionale:**

Il sistema consente l'esecuzione (errata) dell'analisi anche quando "Multiple" è attivo con un solo progetto.

- **Root cause di progettazione:**

Non è presente alcun controllo che vincoli "Resume" all'uso effettivo della modalità "Multiple" con più progetti.

Conclusione:

Il test fallisce perché una configurazione non valida viene accettata senza segnalazioni e porta ad un'analisi fallimentare.

Soluzione:

Introdotta una validazione preliminare che impedisca l'attivazione della modalità "Multiple" quando il numero di progetti selezionati è inferiore a due, bloccando l'avvio dell'analisi e visualizzando un messaggio esplicativo:

- conta i progetti (directory) nella cartella di input;
- esclude le directory speciali "output" e "execution_log.txt";
- se ci sono meno di 2 progetti, stampa l'errore "Multiple mode isn't available with only one project" e interrompe (return);
- previene l'esecuzione di analisi inutili in Multiple mode.

Nome del test:	test_TC_1_25
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità sequenziale nonostante le modalità "Multiple", "Parallel" e "Resume" siano attive in presenza di un progetto contenente un file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando "Multiple", "Parallel" e "Resume" sono attive, su un progetto contenente un file Python. In particolare, il test verifica che: • la configurazione venga bloccata; • venga fornito un messaggio esplicativo all'utente. Il fallimento è dovuto a: • Root cause funzionale: Il sistema consente l'esecuzione (errata) dell'analisi anche quando "Multiple" è attivo con un solo progetto. • Root cause di progettazione: L'assenza di controlli sulle combinazioni di modalità consente l'esecuzione di configurazioni incoerenti. Conclusione: Il test fallisce perché una configurazione non valida viene accettata senza segnalazioni e porta ad un'analisi fallimentare.	
Soluzione:	
Introdotta una validazione preliminare che impedisca l'attivazione della modalità "Multiple" quando il numero di progetti selezionati è inferiore a due, bloccando l'avvio dell'analisi e visualizzando un messaggio esplicativo: • conta i progetti (directory) nella cartella di input; • esclude le directory speciali "output" e "execution_log.txt"; • se ci sono meno di 2 progetti, stampa l'errore e interrompe (return);	



- previene l'esecuzione di analisi inutili in Multiple mode.

Nome del test:	test_TC_1_26
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità sequenziale nonostante le modalità "Parallel" e "Resume" siano attive in presenza di un progetto contenente un file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando "Parallel" e "Resume" sono attive in assenza della modalità "Multiple", su un progetto contenente un file Python. In particolare, il test verifica che: • le modalità non supportate vengano segnalate come tali; • l'utente venga informato del comportamento applicato. Il fallimento è dovuto a: • Root cause funzionale: Le modalità "Parallel" e "Resume" vengono ignorate. • Root cause di progettazione: Il sistema degrada automaticamente la configurazione senza produrre alcun messaggio informativo. Conclusione: Il test fallisce perché il comportamento reale dell'applicazione non è comunicato all'utente.	
Soluzione:	
Introdotta la visualizzazione del messaggio informativo: "Warning: Parallel mode and Resume mode are ignored without Multiple mode".	

Nome del test:	test_TC_1_27
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità sequenziale nonostante la modalità "Parallel" sia attiva in presenza di un progetto contenente un file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando "Parallel" è attivo in assenza della modalità "Multiple", su un progetto contenente un file Python. Il test verifica che: • l'analisi non venga eseguita in parallelo; • l'utente venga informato dell'ignorare della modalità. Il fallimento è dovuto a: • Root cause funzionale: L'analisi viene eseguita in modalità sequenziale senza informare l'utente. • Root cause di progettazione: L'assenza di messaggi informativi rende il comportamento non trasparente. Conclusione: Il test fallisce per mancanza di comunicazione verso l'utente.	
Soluzione:	



Introdotta la visualizzazione del messaggio informativo: “Warning: Parallel mode is ignored without Multiple mode”.

Nome del test:	test_TC_1_28
Errore riportato:	Non viene visualizzato alcun messaggio; l’analisi viene eseguita in modalità sequenziale nonostante la modalità "Resume" sia attiva in presenza di un progetto contenente un file Python.
Causa del problema:	
Il test intende verificare il comportamento dell’applicazione quando "Resume" è attiva in assenza della modalità "Multiple", su un progetto contenente un file Python. In particolare, il test verifica che: • la modalità venga segnalata come non applicabile; • l’analisi venga avviata da zero solo dopo notifica. Il fallimento è dovuto a: • Root cause funzionale: "Resume" viene ignorato e l’utente non viene informato. • Root cause di progettazione: L’assenza di validazioni consente l’uso silenzioso di una modalità non supportata. Conclusione: Il test fallisce perché il sistema non segnala l’incompatibilità della configurazione.	
Soluzione:	
Introdotta la visualizzazione del messaggio informativo: “Warning: Resume mode is ignored without Multiple mode”.	

Nome del test:	test_TC_1_29
Errore riportato:	Non viene visualizzato alcun messaggio; l’analisi viene eseguita nonostante la modalità "Multiple" sia attiva in presenza di un progetto contenente più file di cui almeno uno Python.
Causa del problema:	
Il test intende verificare l’uso della modalità "Multiple" con un solo progetto e più file di cui almeno uno Python. Il test verifica che: • la configurazione venga rifiutata; • l’analisi non venga avviata. Il fallimento è dovuto a: • Root cause funzionale: Il sistema consente l’esecuzione dell’analisi. • Root cause di progettazione: Mancano controlli di coerenza tra numero di progetti e modalità di esecuzione. Conclusione: Il test fallisce perché una configurazione non valida viene accettata.	
Soluzione:	
Introdotta una validazione preliminare che impedisca l’attivazione della modalità "Multiple" quando il numero di progetti selezionati è inferiore a due, bloccando l’avvio dell’analisi e visualizzando un messaggio esplicativo: • conta i progetti (directory) nella cartella di input;	



- esclude le directory speciali "output" e "execution_log.txt";
- se ci sono meno di 2 progetti, stampa l'errore "Multiple mode isn't available with only one project" e interrompe (return);
- previene l'esecuzione di analisi inutili in Multiple mode.

Nome del test:	test_TC_1_30
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita nonostante le modalità "Multiple" e "Parallel" siano attive in presenza di un progetto contenente più file di cui almeno uno Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando viene richiesta l'esecuzione in modalità "Multiple" e "Parallel" su un singolo progetto contenente più file di cui almeno uno Python. In particolare, il test verifica che: • la modalità "Multiple" non venga consentita in presenza di un solo progetto; • l'analisi non venga avviata con una configurazione logicamente incoerente; • venga restituito un messaggio esplicativo che segnali l'invalidità della configurazione selezionata. Il fallimento è dovuto a: • Root cause funzionale: Il sistema consente l'avvio dell'analisi anche quando la modalità "Multiple" è attiva con un solo progetto, ignorando il vincolo semantico associato a tale modalità. • Root cause di progettazione: Non è implementato un controllo preliminare che verifichi la coerenza tra il numero di progetti selezionati e l'attivazione congiunta delle modalità "Multiple" e "Parallel". Conclusione: Il test fallisce perché il sistema accetta ed esegue una configurazione non valida senza produrre alcun avviso o blocco dell'esecuzione.	
Soluzione:	
Introdotta una validazione preliminare che impedisca l'attivazione della modalità "Multiple" quando il numero di progetti selezionati è inferiore a due, bloccando l'avvio dell'analisi e visualizzando un messaggio esplicativo: • conta i progetti (directory) nella cartella di input; • esclude le directory speciali "output" e "execution_log.txt"; • se ci sono meno di 2 progetti, stampa l'errore "Multiple mode isn't available with only one project" e interrompe (return); • previene l'esecuzione di analisi inutili in Multiple mode.	

Nome del test:	test_TC_1_31
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità sequenziale nonostante le modalità "Multiple" e "Resume" siano attive in presenza di un progetto contenente più file di cui almeno uno Python.
Causa del problema:	



Il test intende verificare il comportamento dell'applicazione quando vengono attivate le modalità "Multiple" e "Resume" in presenza di un solo progetto contenente più file di cui almeno uno Python. In particolare, il test verifica che:

- l'uso della modalità "Resume" venga consentito solo in presenza di più progetti;
- una configurazione non valida venga intercettata prima dell'avvio dell'analisi;
- venga fornito un messaggio esplicativo all'utente.

Il fallimento è dovuto a:

- **Root cause funzionale:**
La modalità "Resume" viene ignorata e l'analisi viene avviata come nuova esecuzione.
- **Root cause di progettazione:**
Non è presente un controllo che vincoli l'uso di "Resume" alla presenza effettiva della modalità "Multiple" con più progetti.

Conclusione:

Il test fallisce perché il sistema ignora silenziosamente una modalità non applicabile, senza segnalare l'invalidità della configurazione.

Soluzione:

Introdotta una validazione preliminare che impedisca l'attivazione della modalità "Multiple" quando il numero di progetti selezionati è inferiore a due, bloccando l'avvio dell'analisi e visualizzando un messaggio esplicativo:

- conta i progetti (directory) nella cartella di input;
- esclude le directory speciali "output" e "execution_log.txt";
- se ci sono meno di 2 progetti, stampa l'errore "Multiple mode isn't available with only one project" e interrompe (return);
- previene l'esecuzione di analisi inutili in Multiple mode.

Nome del test:	test_TC_1_32
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita nonostante le modalità "Multiple", "Parallel" e "Resume" siano attive in presenza di un progetto contenente più file di cui almeno uno Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando vengono attivate contemporaneamente le modalità "Multiple", "Parallel" e "Resume" in presenza di un solo progetto contenente più file di cui almeno uno Python. In particolare, il test verifica che: • la modalità "Multiple" non venga consentita con un solo progetto; • l'eventuale incompatibilità tra modalità venga segnalata prima dell'avvio dell'analisi. Il fallimento è dovuto a: • Root cause funzionale: L'analisi viene eseguita da zero in modalità multiple producendo errore nell'output. • Root cause di progettazione: Non è implementato un controllo di coerenza tra numero di progetti selezionati e combinazione delle modalità "Multiple", "Parallel" e "Resume". Conclusione: Il test fallisce perché il sistema consente l'esecuzione con una configurazione logicamente non valida.	



Soluzione:

Introdotta una validazione preliminare che impedisca l'attivazione della modalità "Multiple" quando il numero di progetti selezionati è inferiore a due, bloccando l'avvio dell'analisi e visualizzando un messaggio esplicativo:

- conta i progetti (directory) nella cartella di input;
- esclude le directory speciali "output" e "execution_log.txt";
- se ci sono meno di 2 progetti, stampa l'errore "Multiple mode isn't available with only one project" e interrompe (return);
- previene l'esecuzione di analisi inutili in Multiple mode.

Nome del test:	test_TC_1_33
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità sequenziale da zero nonostante le modalità "Parallel" e "Resume" siano attive in presenza di un progetto contenente più file di cui almeno uno Python.

Causa del problema:

Il test intende verificare il comportamento dell'applicazione quando vengono attivate le modalità "Parallel" e "Resume" in assenza della modalità "Multiple", su un singolo progetto contenente più file di cui almeno uno Python. In particolare, il test verifica che:

- "Parallel" venga applicato solo in presenza della modalità "Multiple";
- "Resume" venga applicato solo in presenza della modalità "Multiple" e in assenza della modalità "Parallel";
- l'utente venga informato dell'eventuale ignorare delle modalità non supportate.

Il fallimento è dovuto a:

- **Root cause funzionale:**
Le modalità "Parallel" e "Resume" vengono ignorate e l'analisi viene eseguita in modalità sequenziale.
- **Root cause di progettazione:**
Il sistema effettua una degradazione automatica della configurazione senza produrre alcun messaggio informativo.

Conclusione:

Il test fallisce perché la configurazione selezionata non viene validata e il comportamento effettivo dell'applicazione non è reso esplicito.

Soluzione:

Introdotta la visualizzazione del messaggio informativo: "Warning: Parallel mode and Resume mode are ignored without Multiple mode".

Nome del test:	test_TC_1_34
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità sequenziale nonostante la modalità "Parallel" sia attiva in presenza di un progetto contenente più file di cui almeno uno Python.

Causa del problema:



Il test intende verificare il comportamento dell'applicazione quando la modalità "Parallel" è attiva in assenza della modalità "Multiple", su un singolo progetto contenente più file di cui almeno uno Python. Il test verifica che:

- la modalità "Parallel" non venga applicata senza "Multiple";
- venga fornito un messaggio informativo all'utente.

Il fallimento è dovuto a:

- **Root cause funzionale:**
L'analisi viene eseguita correttamente ma in modalità sequenziale.
- **Root cause di progettazione:**
L'assenza di un controllo esplicito sulla dipendenza tra "Parallel" e "Multiple" impedisce la segnalazione dell'incompatibilità.

Conclusione:

Il test fallisce per mancanza di trasparenza nella gestione delle modalità non applicabili.

Soluzione:

Introdotta la visualizzazione del messaggio informativo: "Warning: Parallel mode is ignored without Multiple mode".

Nome del test:	test_TC_1_35
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità sequenziale da zero nonostante la modalità "Resume" sia attiva in presenza di un progetto contenente più file di cui almeno uno Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando la modalità "Resume" è attiva in assenza della modalità "Multiple", su un singolo progetto contenente più file di cui almeno uno Python. In particolare, il test verifica che: • "Resume" venga consentito solo in modalità multi-progetto; • l'utente venga informato dell'incompatibilità. Il fallimento è dovuto a: • Root cause funzionale: "Resume" viene ignorato e l'analisi viene eseguita come nuova esecuzione. • Root cause di progettazione: Non è implementato un controllo che vincoli "Resume" alla presenza della modalità "Multiple". Conclusione: Il test fallisce perché una modalità non applicabile viene ignorata senza notifica.	
Soluzione:	
Introdotta la visualizzazione del messaggio informativo: "Warning: Resume mode is ignored without Multiple mode".	

Nome del test:	test_TC_1_36
Errore riportato:	Non viene visualizzato alcun messaggio; l'analisi viene eseguita in modalità sequenziale da zero nonostante le modalità "Parallel"



	e "Resume" siano attive in presenza di più progetti contenenti un solo file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando vengono attivate le modalità "Parallel" e "Resume" in presenza di più progetti contenenti un solo file Python, ma con "Multiple" disattivato. In particolare, il test verifica che: • "Parallel" e "Resume" non vengano applicati senza "Multiple"; • venga prodotto un messaggio informativo che segnali la configurazione non valida. Il fallimento è dovuto a: • Root cause funzionale: Le modalità vengono ignorate e l'analisi procede sequenzialmente. • Root cause di progettazione: Non è presente una validazione che imponga l'attivazione di "Multiple" come prerequisito per "Parallel" e "Resume". Conclusione: Il test fallisce perché la dipendenza tra modalità non è formalmente verificata dal sistema.	
Soluzione:	
Introdotta la visualizzazione del messaggio informativo: "Warning: Parallel mode and Resume mode are ignored without Multiple mode".	

Nome del test:	test_TC_1_37
Errore riportato:	Non viene visualizzato alcun messaggio di errore o avviso; l'analisi viene eseguita in modalità sequenziale da zero nonostante le modalità "Parallel" e "Resume" siano attive in presenza di più progetti contenenti più file di cui almeno uno Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando vengono attivate le modalità "Parallel" e "Resume" in presenza di più progetti contenenti più file di cui almeno uno Python ma con "Multiple" disattivato. In particolare, il test verifica che: • "Parallel" e "Resume" non vengano applicati senza "Multiple"; • venga prodotto un messaggio informativo che segnali la configurazione non valida. Il fallimento è dovuto a: • Root cause funzionale: Le modalità vengono ignorate e l'analisi procede sequenzialmente. • Root cause di progettazione: Non è presente una validazione che imponga l'attivazione di "Multiple" come prerequisito per "Parallel" e "Resume". Conclusione: Il test fallisce perché la dipendenza tra modalità non è formalmente verificata dal sistema.	
Soluzione:	
Introdotta la visualizzazione del messaggio informativo: "Warning: Parallel mode and Resume mode are ignored without Multiple mode".	



Nome del test:	test_TC_1_38
Errore riportato:	Non viene visualizzato alcun messaggio di errore o avviso; l'analisi viene eseguita in modalità sequenziale da zero nonostante la modalità "Resume" sia attiva in presenza di più progetti contenenti un solo file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando "Resume" è attivo su più progetti contenenti un solo file Python ma "Multiple" è disattivato. In particolare, il test verifica che: • "Resume" venga applicato solo in modalità multi-progetto esplicita; • venga fornito un messaggio di incompatibilità. Il fallimento è dovuto a: • Root cause funzionale: "Resume" viene ignorato e l'analisi viene eseguita integralmente. • Root cause di progettazione: Non è implementato un vincolo che imponga "Multiple" come prerequisito per "Resume". Conclusione: Il test fallisce perché una modalità non supportata viene ignorata senza segnalazione.	
Soluzione:	
Introdotta la visualizzazione del messaggio informativo: "Warning: Resume mode is ignored without Multiple mode".	

Nome del test:	test_TC_1_39
Errore riportato:	Non viene visualizzato alcun messaggio di errore o avviso; l'analisi viene eseguita in modalità sequenziale da zero nonostante la modalità "Resume" sia attiva in presenza di più progetti contenenti almeno un file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando "Resume" è attivo su più progetti con più file di cui almeno uno Python ma "Multiple" è disattivato. In particolare, il test verifica che: • "Resume" venga applicato solo in modalità multi-progetto esplicita; • venga fornito un messaggio di incompatibilità. Il fallimento è dovuto a: • Root cause funzionale: "Resume" viene ignorato e l'analisi viene eseguita integralmente. • Root cause di progettazione: Non è implementato un vincolo che imponga "Multiple" come prerequisito per "Resume". Conclusione: Il test fallisce perché una modalità non supportata viene ignorata senza segnalazione.	
Soluzione:	
Introdotta la visualizzazione del messaggio informativo: "Warning: Resume mode is ignored without Multiple mode".	



Nome del test:	test_TC_1_40
Errore riportato:	Non viene visualizzato alcun messaggio di errore o avviso; l'analisi viene eseguita in modalità sequenziale nonostante la modalità "Parallel" sia attiva in presenza di più progetti contenenti un solo file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando "Parallel" è attivo su più progetti contenenti un solo file Python ma "Multiple" è disattivato. In particolare, il test verifica che: - la modalità "Parallel" venga applicata solo in presenza della modalità "Multiple"; - la configurazione non valida venga intercettata prima dell'avvio dell'analisi; - venga visualizzato un messaggio che segnali l'incompatibilità tra modalità selezionate. Il fallimento è dovuto a: Root cause funzionale: La modalità "Parallel" viene ignorata e l'analisi viene eseguita in modalità sequenziale. Root cause di progettazione: Non è implementato un controllo esplicito che imponga la modalità "Multiple" come prerequisito per l'attivazione della modalità "Parallel". Conclusione: Il test fallisce perché il sistema consente l'esecuzione con una configurazione incoerente senza fornire alcuna segnalazione all'utente.	
Soluzione:	
Introdotta la visualizzazione del messaggio informativo: "Warning: Parallel mode is ignored without Multiple mode".	

Nome del test:	test_TC_1_41
Errore riportato:	Non viene visualizzato alcun messaggio di errore o avviso; l'analisi viene eseguita in modalità sequenziale nonostante la modalità "Parallel" sia attiva in presenza di più progetti contenenti più file Python.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando vengono selezionati più progetti con almeno un file Python, la modalità "Parallel" è attiva ma la modalità "Multiple" è disattivata. In particolare, il test verifica che: - la modalità "Parallel" venga applicata solo in presenza della modalità "Multiple"; - la configurazione non valida venga intercettata prima dell'avvio dell'analisi; - venga visualizzato un messaggio che segnali l'incompatibilità tra modalità selezionate. Il fallimento è dovuto a: Root cause funzionale: La modalità "Parallel" viene ignorata e l'analisi viene eseguita in modalità sequenziale. Root cause di progettazione: Non è implementato un controllo esplicito che imponga la modalità "Multiple" come prerequisito per l'attivazione della modalità "Parallel".	



Conclusione:

Il test fallisce perché il sistema consente l'esecuzione con una configurazione incoerente senza fornire alcuna segnalazione all'utente.

Soluzione:

Introdotta la visualizzazione del messaggio informativo: "Warning: Parallel mode is ignored without Multiple mode".

3.3 CR01

La Change Request **CR01** è una richiesta di manutenzione **additiva** che estende il sistema con una modalità di detection basata su **Large Language Model (LLM)**, mantenendo **invariata** la pipeline standard di analisi statica. L'obiettivo principale è introdurre una detection "prompt-driven" e configurabile, attivabile dall'utente in modo **opzionale**, evitando impatti sul comportamento esistente quando la modalità LLM non è selezionata.

Dal punto di vista funzionale, CR01 abilita:

- la **selezione del provider** LLM (locale o via API);
- la **selezione degli smell** su cui applicare la detection LLM;
- la **gestione strutturata** di smell e prompt attraverso un catalogo persistente;
- il **Prompt Engineering**: definizione e modifica di prompt smell-specific, gestione draft/default, test iterativo e promozione controllata del draft a default;
- strumenti dedicati per la manutenzione del catalogo (**Code Smell Management**)

Dal punto di vista architetturale, l'estensione è stata realizzata secondo principi di **separazione delle responsabilità** e **disaccoppiamento**, con l'introduzione di componenti dedicate alla persistenza del catalogo e all'astrazione dei provider, così da rendere la soluzione estendibile verso nuovi provider e nuovi smell senza necessità di refactoring invasivi.

3.4 Prompt Engineering

Nell'ambito della Fase 1 della CR01 è stata implementata integralmente la funzionalità **UC02 - Prompt Engineering** (riferimento al documento *Use Case*), concepita come ambiente controllato di sperimentazione per la detection di ML-specific code smell tramite Large Language Model.

Questa fase non introduce ancora l'integrazione diretta della detection LLM nel flusso principale della GUI "Run", ma realizza l'infrastruttura architetturale e l'interfaccia necessarie per:

- definire e modificare prompt smell-specific;
- testare iterativamente tali prompt su un progetto reale;
- distinguere tra versioni temporanee (draft) e versioni di default;
- validare il comportamento dell'LLM prima dell'integrazione nella pipeline ufficiale.



L'obiettivo della fase è costruire un ambiente di validazione iterativa che consenta allo sviluppatore di comprendere, valutare e migliorare progressivamente il comportamento dell'LLM.

3.4.1 Architettura del sottosistema

L'implementazione è stata progettata seguendo un principio fondamentale: la modalità LLM deve essere additiva e non invasiva rispetto alla pipeline AST-based esistente.

A tal fine:

- nessuna classe della pipeline di analisi statica è stata modificata;
- il nuovo codice è stato isolato nel package `llm_detection`;
- la GUI di Prompt Engineering è stata mantenuta separata dalla GUI principale di esecuzione, nella GUI dedicata `prompt_engineering/prompt_engineering_gui`.

Questa scelta garantisce che CodeSmile rimanga completamente utilizzabile nella modalità tradizionale, anche in assenza di configurazioni LLM.

L'architettura introdotta si articola in tre livelli distinti:

1. Persistenza strutturata di smell, prompt e provider
2. Astrazione dei provider LLM
3. Orchestrazione della detection LLM
4. Interfaccia grafica di Prompt Engineering

La separazione tra questi livelli consente estensibilità futura senza necessità di refactoring invasivo.

3.4.2 Persistenza strutturale del catalogo LLM

Per supportare la modalità LLM è stato introdotto un catalogo persistente in formato JSON che rappresenta l'unica fonte di verità per:

- elenco degli smell disponibili in modalità LLM;
- prompt di default associati a ciascuno smell;
- prompt temporanei (draft) utilizzati in fase di sperimentazione;
- provider LLM configurati (locali e API);

La distinzione tra **prompt temporaneo (draft)** e **prompt di default** è centrale nella logica implementata.

Il draft rappresenta uno spazio di sperimentazione: può essere modificato liberamente e testato senza compromettere la baseline ufficiale.

Il prompt di default, invece, rappresenta la versione validata e stabile, utilizzata nella detection effettiva.

La promozione da draft a default è un'operazione esplicita e controllata.

Questo meccanismo risponde direttamente al requisito della CR01 di separare sperimentazione e configurazione stabile.

La gestione del catalogo è incapsulata in un service layer dedicato che si occupa di:



- validazione degli input;
- salvataggio atomico;
- promozione del draft;
- caricamento coerente dei dati.

In questo modo la GUI non manipola direttamente il file JSON, ma opera tramite un livello di astrazione coerente con i principi di separazione delle responsabilità.

3.4.3 Astrazione dei provider LLM

Uno dei vincoli architetturali della CR01 è il supporto sia a LLM locali sia a provider API.

Per evitare accoppiamenti forti con una tecnologia specifica, è stata introdotta un'astrazione dedicata ai provider LLM. L'orchestrazione della detection dipende esclusivamente da questa interfaccia astratta, mentre le implementazioni concrete (locale, API, mock) sono intercambiabili.

Attualmente:

- è pienamente funzionante il provider locale basato su Ollama;
- è presente uno stub per provider API futuri;
- è disponibile un provider mock per test deterministici.

Questa scelta è coerente con il Bridge Pattern: l'orchestratore rappresenta l'astrazione, mentre i provider costituiscono le implementazioni variabili.

In questo modo, l'aggiunta futura di OpenAI, Claude o Gemini non richiederà modifiche alla logica di detection.

3.4.4 Orchestrazione della detection LLM

È stata introdotta una componente centrale di orchestrazione con il compito di:

- costruire il prompt finale;
- invocare il provider selezionato;
- normalizzare la risposta;
- trasformare il risultato in una struttura coerente con la reportistica.

La costruzione del prompt segue una logica precisa:

- il prompt smell-specific contiene esclusivamente definizione e regole del code smell;
- il contratto di output JSON è imposto centralmente dall'orchestratore;
- il codice sorgente viene fornito con numerazione esplicita delle linee.

Questa scelta è strategica: lo schema di output non è delegato allo smell prompt, ma è garantito a livello di sistema. Ciò evita incoerenze tra smell diversi e riduce il rischio di errori di formato.

3.4.5 Normalizzazione e gestione dell'output LLM

La gestione dell'output LLM rappresenta uno degli aspetti più delicati dell'intera implementazione.



Un LLM può:

- non rispettare il formato richiesto;
- restituire JSON parziale;
- generare testo libero;
- rifiutare la richiesta.

Per affrontare questo problema sono state introdotte due modalità di normalizzazione:

Modalità STRICT

Accetta esclusivamente risposte che rispettano esattamente lo schema previsto. Qualsiasi deviazione produce zero risultati. Questa modalità è pensata per la detection ufficiale.

Modalità SALVAGE

Utilizzata nel Prompt Engineering, tenta un recupero best-effort:

- effettua parsing robusto del JSON;
- intercetta strutture alternative;
- genera un finding diagnostico in caso di schema invalido;
- conserva sempre la risposta grezza (raw response).

Questa scelta è deliberata: il sistema non “interpreta” semanticamente risposte ambigue, ma fornisce allo sviluppatore strumenti per analizzarle consapevolmente.

La responsabilità del miglioramento del prompt rimane quindi esplicitamente a carico allo sviluppatore.

3.4.6 Interfaccia grafica di Prompt Engineering

È stata implementata una GUI dedicata, separata dalla GUI principale.

L'interfaccia consente di:

- selezionare uno smell dal catalogo;
- visualizzare il prompt di default o quello temporaneo;
- modificare il draft;
- salvare il draft;
- promuovere il draft a default;
- selezionare un provider LLM locale;
- selezionare cartelle di input e output;
- eseguire un test su uno o più file Python;
- monitorare lo stato di esecuzione;
- interrompere il test;
- consultare sia i risultati strutturati (CSV) sia le risposte grezze (JSONL).



L'esecuzione avviene in thread separato per evitare il blocco dell'interfaccia.
È stato implementato un meccanismo di heartbeat che aggiorna lo stato e il tempo trascorso.

La GUI non tenta di interpretare semanticamente i risultati.
Il raw output è sempre salvato per consentire un'analisi manuale dettagliata.

Il Prompt Engineering è quindi concepito come **strumento di validazione iterativa**, non come detection definitiva.

3.5 Code Smell Management

È stata implementata la gestione strutturale del catalogo tramite il modulo **Code Smell Management**. Questa componente realizza l'interfaccia operativa per tre specifici Use Case, necessari per la gestione del catalogo degli smell supportati dal sistema:

- **UC – Add Code Smell:** Inserimento di nuovi smell nel catalogo.
- **UC – Modify Code Smell:** Aggiornamento delle descrizioni.
- **UC – Remove Code Smell:** Eliminazione di smell dal catalogo.

L'obiettivo di questa fase è disaccoppiare la definizione teorica dello smell (il *cosa*) dalla sua implementazione tecnica nel prompt (il *come*). Le operazioni effettuate qui agiscono direttamente sulla persistenza JSON, garantendo che le modifiche siano immediatamente disponibili trasversalmente a tutti i moduli del sistema (Prompt Engineering, GUI di esecuzione).

3.5.1 UC – Add Code Smell (Definizione e Validazione)

L'aggiunta di un nuovo code smell è un'operazione critica che stabilisce l'identità univoca dell'entità nel sistema. L'interazione avviene tramite una finestra di dialogo modale dedicata (AddSmellDialog), progettata per garantire l'integrità dei dati prima della persistenza. Il sistema applica regole di validazione rigorose in fase di creazione:

- **Generazione dell'Identificativo:** A partire dal nome visualizzato (es. "God Class"), il sistema genera automaticamente uno *slug* normalizzato (es. god-class) che funge da chiave primaria immutabile per il catalogo.
- **Controllo di Univocità:** Viene impedita la creazione di duplicati che potrebbero causare conflitti nella generazione dei report.
- **Obbligatorietà dei Metadati:** Non è consentito creare smell privi di descrizione.

3.5.2 UC – Modify Code Smell (Evoluzione della Definizione)

Il sistema supporta l'evoluzione della conoscenza sugli smell permettendo la modifica delle loro descrizioni. L'architettura distingue nettamente tra dati immutabili e mutabili per preservare la coerenza storica:

- L'**ID** e il nome dello smell sono rigorosamente **immutabili**. Questo garantisce che i report generati in passato rimangano tracciabili e che i collegamenti con i prompt salvati non vengano interrotti.
- La **Descrizione** è **modificabile**. Questo permette allo sviluppatore di raffinare la definizione teorica dello smell man mano che la comprensione del problema



evolve. L'operazione di salvataggio (`_on_save_changes`) persiste le modifiche in modo atomico sul file JSON del catalogo.

3.5.3 UC – Remove Code Smell (Manutenzione del Catalogo)

La rimozione di uno smell richiede conferma esplicita da parte dell'utente per prevenire cancellazioni accidentali. A livello architetturale, la rimozione:

1. Elimina la voce dal catalogo JSON caricato in memoria.
2. Aggiorna immediatamente il file su disco tramite il `LLMCatalogService`.
3. Sincronizza l'interfaccia grafica rimuovendo l'elemento dalla lista di selezione e pulendo i pannelli di dettaglio. Questa operazione garantisce che non rimangano riferimenti orfani a smell non più supportati nel sistema.

3.5.4 Interfaccia Grafica e State Management

L'interfaccia (`ManageCodeSmellsGUI`) orchestra questi tre Use Case attraverso un pannello di controllo unificato. La logica di presentazione implementa un controllo rigoroso dello stato (`State Management`) per guidare l'utente:

- **Contesto Attivo:** I comandi di Modifica e Rimozione sono abilitati esclusivamente quando uno smell è selezionato.
- **Feedback Visivo:** I campi identificativi (ID/Nome) sono presentati in stato `readonly` post-creazione per rinforzare il concetto di immutabilità dell'identificatore.
- **Sincronizzazione:** Ogni operazione (Aggiunta, Modifica, Rimozione) innesca un `refresh` immediato del catalogo e della vista, garantendo che l'utente operi sempre sullo stato più aggiornato dei dati.

3.6 Code Smell Detection With LLM

Nell'ambito della CR01 è stata implementata l'integrazione della **detection LLM** nella GUI principale di CodeSmile (Use Case **UC01 – Code Smell Detection static + LLM**). L'obiettivo dell'intervento è estendere il flusso di esecuzione esistente introducendo una modalità LLM **opzionale e non invasiva**, mantenendo invariato il comportamento della pipeline standard basata su analisi statica.

Le modifiche realizzano in particolare:

- attivazione/disattivazione della modalità LLM direttamente dalla finestra "Run" tramite un controllo dedicato;
- selezione del **tipo di provider** (Local/API) e del provider specifico tramite controlli GUI;
- selezione **multi-smell** coerente con UC01;
- esecuzione della detection LLM **a valle** dell'analisi statica, con salvataggio dei risultati in un report CSV dedicato.

3.6.1 Principi di implementazione e retrocompatibilità

L'integrazione è stata progettata secondo i seguenti principi:



- **Additività e non invasività:** la modalità LLM è disabilitata di default e viene abilitata esplicitamente dall'utente tramite checkbox;
- **Separazione delle responsabilità:** la GUI non accede direttamente al file JSON del catalogo ma si appoggia a un service layer (**LLMCatalogService**);
- **Disaccoppiamento tramite astrazione:** l'esecuzione LLM utilizza l'orchestratore (**LLMOrchestrator**) e i provider concreti (**LocalLLMProvider** / **ApiLLMProvider**) secondo un'impostazione coerente con il Bridge Pattern, evitando accoppiamenti con tecnologie specifiche;
- **Retrocompatibilità:** la pipeline standard e le modalità operative pre-esistenti rimangono invariate; l'estensione del flusso è realizzata tramite parametri opzionali e controlli attivati solo se LLM-mode è selezionata.

A valle dell'implementazione, la modalità tradizionale resta pienamente utilizzabile anche in assenza di configurazioni LLM e l'integrazione LLM non altera l'esecuzione standard se non attivata.

3.6.2 Modifiche introdotte nella GUI "Run"

Le modifiche principali sono state concentrate nel file

gui/code_smell_detector_gui.py, con l'introduzione di un blocco di configurazione LLM visualizzato solo quando richiesto dall'utente.

Nuovi controlli GUI

- **Checkbox "Abilita Detection con LLM":** abilita/disabilita la modalità LLM e gestisce la visibilità dei controlli dedicati tramite una funzione di toggle (es. `toggle_llm_controls()`).
- **Frame "LLM Configuration"** (nascosto di default): contiene i controlli di selezione e viene mostrato solo se la modalità LLM è attiva.
- **Selezione Provider Type (Local/API)** tramite radio button: abilita il filtraggio dei provider disponibili.
- **Provider Combobox** (readonly): consente la selezione di un provider specifico tra quelli configurati nel catalogo, mostrando un nome descrittivo (`display_name`).
- **Smell Listbox multi-selezione** con scrollbar: permette di selezionare uno o più smell da analizzare tramite LLM.

Caricamento e popolamento dati

All'avvio della GUI viene eseguito il caricamento del catalogo LLM (es. `load_llm_data()`), che popola:

- l'elenco provider filtrato per tipo (Local/API);
- l'elenco degli smell disponibili per la detection LLM (abilitati e correttamente configurati nel catalogo).

3.6.3 Validazioni e mapping con UC01

La logica di avvio (es. `run_program()`) è stata estesa con validazioni specifiche attivate solo in presenza di LLM-mode, in coerenza con UC01:



- **validazione selezione provider:** blocca l'avvio della fase LLM se l'utente non seleziona un provider;
- **validazione selezione smell:** blocca l'avvio della fase LLM se l'utente non seleziona alcuno smell.

Le segnalazioni di errore/avviso relative a tali validazioni vengono gestite tramite **output console** (print), in coerenza con la codebase esistente e senza introdurre finestre modali bloccanti.

Inoltre, la configurazione selezionata dall'utente viene mappata da valori user-friendly (display_name) a identificativi tecnici (provider_id, smell_id), necessari per l'invocazione dell'orchestratore.

3.6.4 Esecuzione della detection LLM e produzione output

La detection LLM è integrata come fase **successiva** alla detection statica, preservando ordine e comportamento della pipeline esistente. Al termine dell'analisi standard, la GUI invoca la fase LLM (es. `_run_llm_detection()`) solo se:

- la modalità LLM è attiva,
- è stato selezionato un provider valido,
- è stata effettuata almeno una selezione di smell.

Orchestrazione e provider

La fase LLM:

1. recupera la configurazione del provider dal catalogo;
2. istanzia il provider concreto (**LocalLLMProvider** o **ApiLLMProvider**) in base al tipo selezionato;
3. raccoglie i file Python dal path di input e costruisce i target di detection;
4. invoca l'**LLMOrchestrator** passandogli provider e catalogo;
5. riceve risultati e statistiche (file processati, prompt inviati, numero di findings).

Output

I risultati della detection LLM vengono esportati in un **CSV dedicato** nella cartella di output tramite una routine di salvataggio (es. `_save_llm_findings()`), mantenendo separati i risultati LLM dalla reportistica della detection statica. Nel repository, il file viene salvato con nome `llm_detection_results.csv`.